

Graphical Abstract

Context-Aware Dynamic Tool Resolution for Deadline-Constrained Autonomous Agents

Anonymous

Context-Aware Dynamic Tool Resolution for Deadline-Constrained Autonomous Agents

Anonymous

Abstract

Autonomous agents that augment their reasoning with external tools face a deadline-constrained orchestration problem: invoking a tool-rich *Strategic* pipeline can improve decision quality, but it also increases latency, variability, and the risk of catastrophic deadline failure. We formalize this setting as a non-preemptive single-server queue with firm deadlines and a multi-resolution tool menu comprising a fast *Tactical* mode (base survival utility) and a slower *Strategic* mode (survival plus an intelligence bonus). Missing a deadline yields zero utility—a catastrophic outcome in mission-critical applications.

We first establish *Firm-Deadline Tool Control (FTC)*, a feasibility-gating foundation whose exact monotone threshold structure we prove in estimated backlog, deadline budget, and time-margin parameter ε , including a multi-mode generalization to arbitrarily many tool tiers. We then introduce *Context-Aware Dynamic Tool Resolution (CADTR)*, which extends FTC with event-driven dynamic risk adaptation: upon detecting a critical event, CADTR applies a context-dependent ε -shift that tightens the feasibility budget, dynamically sacrificing the Strategic intelligence bonus to guarantee physical survival.

We evaluate CADTR against two reviewer-motivated oracle baselines—a Mode-Aware Oracle with exact per-task queue-wait knowledge, and a Myopic CDF Oracle that maximizes expected on-time utility via Gaussian CDF approximation—both of which possess strictly more information than CADTR. In trace-driven discrete-event simulations calibrated with three LLM backends (Llama 3.1, Qwen 2.5 7B Instruct, Mistral 7B Instruct) under threat-burst scenarios (25% critical tasks, arrival rate λ swept from 0.03 to 0.19 req/s), the mathematically optimal oracles drop up to $\sim 14\%$ of critical tasks, whereas CADTR maintains a practically flat $\sim 3.5\%$ critical miss rate by dynamically throttling Strategic tool usage from 41% down to 5% as load increases.

Keywords: tool-augmented autonomous agents, context-aware orchestration, multi-resolution tools, deadline-constrained scheduling, dynamic risk adaptation, mission-critical AI systems

1. Introduction

Autonomous agents increasingly augment their reasoning with external tools—calculators, retrieval engines, code executors, and structured APIs—to compensate for the limitations of pure model inference [1, 2]. While tool augmentation improves decision quality, it fundamentally changes the serving problem: it turns a single inference into a multi-stage pipeline with external dependencies, higher latency, and substantially more variance. In mission-critical applications—real-time monitoring, autonomous operations, and safety-constrained systems—this variability is not merely inconvenient; a missed deadline can mean catastrophic failure [3].

We study this as a *multi-resolution tool orchestration* problem. Each arriving task must be routed to one of two pipelines: a fast *Tactical* mode that provides base survival utility at low latency, or a slower *Strategic* mode that adds an intelligence bonus (e.g., enriched telemetry, audit logging, deeper contextual reasoning) at the cost of higher and more variable service time. Missing a deadline yields zero utility—a firm, catastrophic outcome. The orchestrator’s challenge is to maximize the aggregate utility delivered on time while protecting critical tasks under congestion.

Novelty relative to classic admission control. Classical deadline scheduling and admission control assume that a job’s service requirement is fixed once admitted. In tool-augmented serving, the orchestrator can *shape the service-time distribution* before execution by selecting a tool-permission profile, trading utility for latency in a way that is directly coupled to queue state. Crucially, classical methods lack *event-level semantic awareness*: they cannot differentiate between a routine task and a mission-critical one. Our main contribution, CADTR, introduces exactly this dimension—a context-dependent risk adaptation that tightens the feasibility budget for critical events, a mechanism with no analogue in standard admission control or feasibility scheduling.

1.1. From FTC to CADTR

We first establish *Firm-Deadline Tool Control (FTC)*, an arrival-epoch feasibility-gating policy that selects between the Strategic (SLOW) and Tac-

tical (FAST) tool pipelines based on predicted deadline feasibility. FTC estimates the waiting time implied by (i) residual in-service work and (ii) queued backlog, then applies a feasibility test that compares the predicted completion time of the Strategic pipeline against the request deadline budget. A single multiplicative time-margin factor, ε , relaxes or tightens this test, yielding an interpretable operating-point parameter for trading on-time utility against deadline-miss rate without changing the underlying model or tool stack.

We then introduce *Context-Aware Dynamic Tool Resolution (CADTR)*, which extends FTC with *event-driven dynamic risk adaptation*. When a critical (high-priority) event arrives, CADTR applies a context-dependent ε -shift that tightens the feasibility budget, biasing the policy toward the Tactical tool and dynamically sacrificing the intelligence bonus to guarantee physical survival. For standard events, CADTR behaves identically to FTC. This minimal mechanism—a single additive shift conditioned on event priority—has an outsized effect: under threat-burst scenarios, it shields critical tasks where mathematically optimal oracle baselines fail catastrophically.

1.2. Trace-driven performance evaluation

To evaluate orchestration policies under realistic latency variability, we use discrete-event simulation driven by empirical end-to-end latency traces collected from a tool-augmented LLM agent. In addition to standard mode-wise trace collection, we perform counterfactual measurements by executing the same prompt under both modes, yielding paired samples that reduce routing-induced selection bias. This enables trace-driven evaluation without requiring a parametric latency model, aligning with recent interest in simulation frameworks for LLM inference that support controlled studies of queueing and scheduling policies under realistic variability [4]. Because FTC consumes only calibrated mode-wise latency summaries rather than model-specific internals, the same evaluation framework can be re-instantiated on another tool-augmented LLM stack by recollecting traces under the corresponding fast/slow execution profiles.

Beyond the two-mode formulation, we also establish in Theorem 4.2 that the same feasibility logic extends to a finite ordered family of execution modes, preserving an exact threshold characterization of the selected mode.

1.3. Contributions

Contributions. This paper makes five contributions:

- **Model:** We formalize deadline-constrained tool-augmented inference as a non-preemptive single-server queue with firm deadlines and a physically grounded multi-resolution utility model: Tactical tools provide base survival utility, Strategic tools add an intelligence bonus, and missed deadlines are catastrophic (zero utility).
- **FTC foundation:** We introduce Firm-Deadline Tool Control (FTC), a backlog-aware feasibility-gating policy that selects among multi-resolution tool pipelines at the arrival epoch.
- **Formal results:** We prove that FTC has an exact monotone threshold structure in estimated backlog, deadline budget, and time-margin ε (Theorem 4.1), and that this structure extends to arbitrarily many ordered tool tiers (Theorem 4.2).
- **CADTR:** We introduce Context-Aware Dynamic Tool Resolution (CADTR), which extends FTC with event-driven dynamic risk adaptation via a priority-dependent ε -shift that shields critical tasks under threat-burst scenarios.
- **Oracle benchmarking:** We evaluate CADTR against a Mode-Aware Oracle (exact per-task queue-wait knowledge) and a Myopic CDF Oracle (expected-utility maximization via Gaussian CDF)—both with strictly more information than CADTR. Under a nine-point load sweep with 25% critical tasks, the oracles drop up to $\sim 14\%$ of critical tasks while CADTR maintains $\sim 3.5\%$.

1.4. Paper organization

Section 2 reviews related work. Section 3 presents the system model, including the multi-resolution utility model. Section 4 defines FTC, establishes its structural properties, and introduces the CADTR extension. Section 5 describes the baseline policies, including the two oracle baselines. Section 6 details the trace-driven simulation and experimental methodology. Section 7 reports results, Section 8 discusses scope and deployment implications, and Section 9 concludes.

2. Related Work

Our work sits at the intersection of (i) tool-augmented LLM agents, (ii) SLO/deadline-aware LLM serving and queue management, and (iii) queue-

ing/control foundations for deadlines and overload protection [5]. We briefly review each area and clarify how our approach differs: FTC and its extension CADTR operate as a *systems-layer* policy that selects among a small set of inference *pipelines* (multi-resolution tool profiles) under firm deadlines, rather than optimizing tool invocation *within* a pipeline or optimizing kernel-level serving mechanics.

2.1. Tool-augmented LLM agents and constrained tool use

Tool-augmented agents motivate our abstraction of multiple inference modes with different quality–latency profiles. ReAct popularized interleaving LM reasoning with external actions, establishing a widely used pattern for tool-using LLM systems [1]. Toolformer demonstrated that language models can learn to invoke tools to improve downstream performance [2]. These works primarily study *content-level* decisions—when and how to call tools to solve a task—and typically optimize task success or answer quality [6, 7]. Existing methods reason about correctness; FTC reasons about schedulability.

Our approach is complementary: it assumes that tool-rich inference improves expected utility but increases latency and variance, and it controls *whether* a request should be admitted to a tool-rich pipeline given the current backlog and the request time budget. Beyond this feasibility gate, CADTR adds a *context-dependent* dimension: the safety margin adapts per-event based on criticality, a mechanism absent from existing tool-augmented agent frameworks.

2.2. SLO and deadline-aware LLM serving

A rapidly growing body of systems work targets SLO attainment in LLM serving by optimizing the *serving stack*—batching, token scheduling, KV (key-value)-cache/memory management, routing, and cluster-level placement [8].

Kernel- and memory-level serving optimizations. Systems such as vLLM improve throughput and latency via efficient KV-cache memory management (PagedAttention) [9]. These techniques reduce per-request cost but do not decide *which inference pipeline* (e.g., tool-permission profile) should run under a firm request TTL.

Prefill-decode and token-level scheduling. Another line of work exploits LLM phase structure (prefill vs. decode) to improve goodput under latency constraints, e.g., via chunking/piggybacking and schedule-aware execution that better balances throughput and tail latency [10]. This class of methods controls *how tokens are scheduled* once a request is admitted, whereas FTC makes a *mode selection/admission* decision *before* execution to cap the latency variance induced by tool use.

Queue management and SLO-aware orchestration. Serving stacks increasingly incorporate explicit queue-management and SLO-driven policies to stabilize latency under interactive workloads [11, 12, 13, 14]. Recent systems further address heterogeneous SLOs with admission control and deadline-style reordering/rejection [15, 16]. FTC is orthogonal: it *selects the induced workload* by choosing between tool-permission pipelines using deadline-feasibility reasoning (queue state + measured mode latencies), and can therefore compose with these schedulers.

Dynamic rescheduling, disaggregation, and distributed serving. Dynamic rescheduling and migration across model instances can reduce tail latency and improve SLO attainment under heterogeneous workloads [17]. Architectural disaggregation (e.g., separating prefill and decode) further improves goodput and latency under load [16], and distributed serving frameworks aim to reduce end-to-end latency at scale [18]. FTC does not change the serving architecture; it changes *which pipeline* is executed given deadline feasibility.

Augmented/agentive serving and tool-workflow scheduling. As tool-augmented and agentive workflows become common, serving systems increasingly optimize end-to-end workflow latency rather than single-call latency [19], e.g., by using program- or lifecycle-aware scheduling and state management [20, 21, 22]. These efforts focus on scheduling *within* an augmented workflow (or across its stages), whereas FTC introduces a lightweight *front-door* policy that gates entry to tool-rich pipelines under firm request deadlines.

Multi-model routing and cascades. A related line of work routes across multiple model options (or uses cascades) to trade cost/quality/latency, often learning routing rules from preference or quality signals [23, 24, 25]. In contrast, FTC routes between *tool-permission pipelines* using *deadline feasibility* derived from queue state, explicitly targeting *deadline-gated* utility.

2.3. Queueing/control foundations: deadlines, abandonment, feasibility, and drift

FTC is motivated by feasibility reasoning in real-time scheduling, where admission and scheduling decisions are expressed in time-budget units relative to deadlines [26, 27]. Our drift-penalty baseline draws on Lyapunov drift and stochastic network optimization, which balance backlog terms against reward/utility-like objectives [28]. Queueing systems with abandonment emphasize overload protection, robust waiting-time estimation, and deadline-like patience constraints [29, 30, 31]. Finally, feedback scheduling connects control-theoretic methods to QoS targets such as miss ratio [32, 33, 34].

2.4. Positioning

Viewed through this lens, FTC is a feasibility-style admission/mode-selection policy specialized to tool-augmented inference. It converts observable congestion (queue length and residual in-service work) together with empirically measured mode latencies into a predicted waiting time, and admits tool-rich inference only when predicted feasible under the request deadline budget. CADTR extends FTC with *event-driven dynamic risk adaptation*: it adjusts the feasibility margin based on event-level semantic context (e.g., critical vs. standard priority), a mechanism with no direct analogue in the classical scheduling, routing, or queueing literature reviewed above. This context-aware dimension distinguishes our contribution from purely load-reactive policies and from static optimization approaches that lack per-event risk adaptation.

3. System Model

We model a tool-augmented LLM orchestration service that selects an inference mode for each arriving request and executes the resulting pipeline on a shared, non-preemptive server. Requests are associated with firm deadlines (TTLs). If a request has not started service before its deadline, it abandons (is dropped). If it starts service before the deadline, it runs to completion, but yields zero utility if it completes after the deadline. This captures interactive settings where late responses provide no value but still consume capacity.

3.1. Arrivals and deadlines

Requests arrive as a Poisson process with rate $\lambda > 0$; let a_k denote the arrival time of request k . Each request has a TTL budget $\delta_k > 0$ and absolute deadline

$$d_k := a_k + \delta_k. \quad (1)$$

We treat $\{\delta_k\}$ as i.i.d. samples from a regime-dependent distribution. The specific deadline regimes and any truncation/clipping used in experiments are defined in Section 6.3.

The Poisson arrival model is a standard approximation for aggregate, user-driven traffic (and remains a useful baseline even under moderate burstiness), and it allows us to isolate the impact of mode selection and queueing delay on deadline misses.

3.2. Queueing dynamics and timing

The system has an infinite-capacity waiting room and a single server. Service is non-preemptive: once a request begins service, it runs to completion. Let b_k denote the service start time of request k (or $b_k = \infty$ if it abandons), and let c_k denote completion time (or $c_k = \infty$ if it abandons). If request k starts service, its completion time is

$$c_k := \begin{cases} b_k + S_k(m_k), & b_k < \infty, \\ \infty, & b_k = \infty, \end{cases} \quad (2)$$

where m_k is the selected mode and $S_k(m_k)$ is the corresponding service time.

We define the waiting time $\omega_k := b_k - a_k$ and end-to-end latency:

$$L_k := c_k - a_k = \omega_k + S_k(m_k). \quad (3)$$

Realism of assumptions. The single-server abstraction captures a shared bottleneck resource (e.g., a constrained GPU/CPU pool or a serialized tool-execution stage); extending the model and policies to explicit multi-server settings is an important direction for future work.

3.3. Inference modes and service times

The orchestrator selects a mode $m_k \in \mathcal{M}$ at the arrival epoch, where $\mathcal{M} = \{\text{FAST}, \text{SLOW}\}$. We refer to these as the *Tactical* and *Strategic* tool pipelines, respectively. The Strategic (SLOW) mode corresponds to a tool-rich pipeline—e.g., one that includes telemetry logging, enriched contextual

reasoning, or full audit trails—that typically yields higher utility but has larger mean latency and variance. The Tactical (FAST) mode restricts tool use to bare-metal execution, reducing latency at the cost of lower utility.

Service time under mode m is modeled as a random variable $S(m)$ with mode-specific distribution F_m . In simulation, we implement F_m using empirical, end-to-end latency traces collected under each mode (Section 6), and sample $S_k(m)$ accordingly.

Estimated mean service times. For policy computations that require a point estimate of service time (e.g., predicted waiting/sojourn time), we use $\hat{s}(m) := \mathbb{E}[S(m)]$ to denote the *estimated mean service time* under mode m . In particular, $\hat{s}(\text{FAST})$ and $\hat{s}(\text{SLOW})$ are the estimated mean service times of the FAST and SLOW pipelines, respectively, computed from the same mode-specific traces (Section 6).¹

3.4. Multi-resolution utility model

We adopt a physically grounded, firm-deadline utility model that captures the multi-resolution tool paradigm. The realized utility of request k is

$$\tilde{U}_k := U_k(m_k) \mathbf{1}\{c_k \leq d_k\}. \quad (4)$$

Requests that abandon ($b_k = \infty$) or complete after the deadline contribute $\tilde{U}_k = 0$, representing catastrophic failure.

The mode-specific success utility $U_k(m)$ reflects the *multi-resolution tool value*:

- **Tactical tool** (FAST): $U_k(\text{FAST}) = u_{\text{base}}$ — base survival utility. The agent completes its core mission.
- **Strategic tool** (SLOW): $U_k(\text{SLOW}) = u_{\text{base}} + u_{\text{intel}}$ — survival plus intelligence bonus (e.g., enriched telemetry, audit trail, deeper analysis).
- **Missed deadline**: $\tilde{U}_k = 0$ — catastrophic failure.

In experiments, we use $u_{\text{base}} = 1.0$ and $u_{\text{intel}} = 0.5$, with a small noise term ($\sigma = 0.02$) for organic variance. This additive, physically grounded structure—where every successful task earns survival credit and Strategic

¹In online deployments, $\hat{s}(m)$ can be updated via a running average or EWMA over recent completions; our experiments use offline estimates from collected traces.

tools provide a bounded bonus on top—replaces the need for ad hoc utility distributions and makes the trade-off between tool resolution and timeliness transparent.

3.5. Observable state, policies, and objectives

Let $\mathbf{X}_k$ denote the observable system state at arrival of request k . Following standard single-server queueing observables, we use

$$\mathbf{X}_k := (R_k, Q_k), \quad (5)$$

where R_k is the residual service time of the job in service (or $R_k = 0$ if the server is idle) and Q_k is the number of jobs waiting in queue (excluding any job in service).

Predicted wait and related quantities. For a *candidate* mode choice $c \in \mathcal{M}$ for request k , we write $\widehat{W}_k^c$ to denote the *predicted waiting time* (predicted time from arrival until service start) that request k would experience if mode c were selected at arrival. This predicted wait is a function of the observable state $\mathbf{X}_k$ and the service-time estimates $\hat{s}(\cdot)$; concretely, it represents the backlog-induced delay implied by the current residual work R_k and the Q_k queued jobs. We also use the corresponding *predicted sojourn time* (predicted end-to-end latency) under candidate mode c ,

$$\widehat{L}_k^c := \widehat{W}_k^c + \hat{s}(c),$$

i.e., predicted waiting plus the estimated mean service time of the selected mode.

A policy π maps the arrival state and request attributes (including δ_k) to a mode choice: $m_k = \pi(\mathbf{X}_k, \delta_k, \dots)$. We evaluate policies using (i) mean on-time utility $\mathbb{E}[\tilde{U}_k]$ and (ii) deadline-miss rate (DMR), defined as the fraction of requests that do not complete by their deadlines:

$$\text{DMR} := \Pr[c_k > d_k]. \quad (6)$$

Here, DMR includes both (i) abandoned (never started) and (ii) late completion (started but finished late). These metrics reflect the core trade-off in deadline-constrained tool use: higher utility from tool-rich inference versus increased miss probability due to longer service and queueing delay.

3.6. Critical and standard task classes

To model mission-critical scenarios with threat bursts, we consider a two-class system with *critical* (high-priority) and *standard* (normal-priority) tasks sharing the same server. Critical tasks arrive as a Bernoulli-thinned fraction of the Poisson stream (25% critical rate in our experiments). The server selects the head-of-line job from the highest non-empty priority class when it becomes idle (FCFS within each class; service remains non-preemptive). Critical tasks receive a utility multiplier of $1.2\times$, reflecting the higher stakes of mission-critical events. We report both per-class and overall metrics in Section 7. This two-class structure is the foundation of the *threat-burst* evaluation, where CADTR’s context-dependent ε -shift provides differentiated risk adaptation for critical tasks.

4. From Feasibility Gating to Context-Aware Dynamic Tool Resolution

This section develops the policy framework in two stages. We first introduce *Firm-Deadline Tool Control (FTC)*, an arrival-epoch feasibility-gating policy that selects between the Strategic (SLOW) and Tactical (FAST) tool pipelines based on predicted deadline feasibility. We then extend FTC to *Context-Aware Dynamic Tool Resolution (CADTR)*, which adds event-driven dynamic risk adaptation for mission-critical scenarios.

4.1. Feasibility gate

At arrival of request k , the observable state is $\mathbf{X}_k = (R_k, Q_k)$, where R_k is the residual service time of the job in service (or 0 if idle) and Q_k is the number of waiting jobs. Let δ_k be the deadline budget for request k . FTC admits request k to the Strategic (SLOW) pipeline only if the predicted completion time fits within the deadline budget (optionally scaled by $(1+\varepsilon)$), i.e., the feasibility test in Eq. (7) holds:

$$\widehat{W}_k + \hat{s}(\text{SLOW}) \leq (1 + \varepsilon) \delta_k, \quad (7)$$

where $\widehat{W}_k$ is the predicted waiting time until service start and $\hat{s}(\text{SLOW})$ is the expected service time of the Strategic pipeline. If (7) holds, FTC selects SLOW (Strategic); otherwise it selects FAST (Tactical).

4.2. Backlog-based waiting-time estimate

We approximate the waiting time using a standard non-preemptive single-server backlog estimator:

$$\widehat{W}_k := \text{EstimateWait}(R_k, Q_k, \hat{s}^{\text{avg}}) := R_k + Q_k \hat{s}^{\text{avg}}. \quad (8)$$

Here $\hat{s}^{\text{avg}}$ is an estimate of the mean service time of a typical queued job. Because queued jobs may be served in either mode, we model this typical mean as a convex combination:

$$\hat{s}^{\text{avg}} := \rho \hat{s}(\text{SLOW}) + (1 - \rho) \hat{s}(\text{FAST}), \quad (9)$$

where $\rho \in [0, 1]$ is a *queue-mix* parameter representing the expected fraction of queued jobs that will be executed in SLOW mode. Substituting (8) into (7), FTC selects SLOW precisely when

$$R_k + Q_k \hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}) \leq (1 + \varepsilon) \delta_k. \quad (10)$$

This explicit affine form is convenient for both implementation and analysis.

The feasibility inequality (10) is a single affine constraint in $(R_k, Q_k, \delta_k, \varepsilon)$. Several structural properties follow immediately from this form.

Observations.

- (O1) If $\varepsilon \leq -1$, FTC selects FAST for every arrival state, since $(1 + \varepsilon)\delta_k \leq 0 < \hat{s}(\text{SLOW})$.
- (O2) For $\varepsilon > -1$, FTC defines an exact backlog threshold $B^*(\delta_k, \varepsilon) = (1 + \varepsilon)\delta_k - \hat{s}(\text{SLOW})$: SLOW is selected iff $B_k := R_k + Q_k \hat{s}^{\text{avg}} \leq B^*$.
- (O3) Equivalently, for fixed (R_k, δ_k) , this yields a queue threshold $q^* = \lfloor B^* / \hat{s}^{\text{avg}} \rfloor$; and for fixed (R_k, Q_k) , a deadline threshold $\delta^* = (R_k + Q_k \hat{s}^{\text{avg}} + \hat{s}(\text{SLOW})) / (1 + \varepsilon)$.

These observations confirm that FTC is an exact workload-threshold policy. The following theorem formalizes the key structural property—monotone comparative statics—which shows that FTC’s behavior is directionally predictable with respect to all system parameters.

Theorem 4.1 (Monotonicity structure of FTC). *Fix $\rho \in [0, 1]$, and let*

$$\hat{s}^{\text{avg}} = \rho \hat{s}(\text{SLOW}) + (1 - \rho) \hat{s}(\text{FAST}),$$

with $\hat{s}(\text{FAST}) > 0$ and $\hat{s}(\text{SLOW}) > 0$. For $\varepsilon > -1$, the FTC decision is nonincreasing in R_k and Q_k , and nondecreasing in δ_k and ε . In particular, if FTC selects SLOW at $(R, Q, \delta, \varepsilon)$ and

$$R' \leq R, \quad Q' \leq Q, \quad \delta' \geq \delta, \quad \varepsilon' \geq \varepsilon > -1,$$

then FTC also selects SLOW at $(R', Q', \delta', \varepsilon')$.

Proof. Suppose FTC selects SLOW at $(R, Q, \delta, \varepsilon)$, so that $R + Q \hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}) \leq (1 + \varepsilon) \delta$. Take any $(R', Q', \delta', \varepsilon')$ with $R' \leq R$, $Q' \leq Q$, $\delta' \geq \delta$, $\varepsilon' \geq \varepsilon > -1$. Since $\hat{s}^{\text{avg}} > 0$, the left-hand side is nonincreasing:

$$R' + Q' \hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}) \leq R + Q \hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}).$$

Since $\delta' > 0$ and $\varepsilon' \geq \varepsilon$, the right-hand side is nondecreasing: $(1 + \varepsilon') \delta' \geq (1 + \varepsilon) \delta' \geq (1 + \varepsilon) \delta$. Combining gives the desired inequality. $\square$

As residual work or queue length grows, the Strategic-tool admission region contracts monotonically; as deadline budget or the margin parameter increases, it expands monotonically. This gives FTC a clean analytical identity and clarifies why it is a stronger comparator than a pure bang-bang queue threshold: FTC switches in deadline units and depends jointly on backlog and request time budget.

The binary fast/slow abstraction is the simplest nontrivial instance of deadline-aware mode gating, but the same feasibility logic is not tied to two modes. In many tool-augmented serving systems, an orchestrator may have access to a finite ordered menu of execution profiles that trade latency for capability more gradually, e.g., increasingly tool-rich or increasingly deliberate pipelines. The next theorem shows that the structural threshold property of FTC extends directly to this setting. The only change is that the scalar queue-mix parameter ρ is replaced by a probability vector over modes, which induces a backlog estimate through the corresponding weighted average service-time summary.

Multi-mode extension.

Theorem 4.2 (Multi-mode extension of FTC). *Fix $J \geq 2$ modes $\mathcal{M} = \{1, \dots, J\}$, indexed so that*

$$0 < \hat{s}_1 \leq \hat{s}_2 \leq \dots \leq \hat{s}_J,$$

where mode 1 is the fastest/lightest mode and mode J is the slowest/richest mode. Let

$$\boldsymbol{\rho} = (\rho_1, \dots, \rho_J), \quad \rho_j \geq 0, \quad \sum_{j=1}^J \rho_j = 1,$$

and define the average queued-job service-time estimate

$$\hat{s}_{avg} := \sum_{j=1}^J \rho_j \hat{s}_j.$$

For an arrival with observable state (R_k, Q_k) satisfying $R_k \geq 0$ and $Q_k \in \mathbb{Z}_{\geq 0}$, TTL $\delta_k > 0$, and margin $\varepsilon \in \mathbb{R}$, define

$$B_k := R_k + Q_k \hat{s}_{avg}.$$

Consider the J -mode FTC rule that selects the richest feasible mode:

$$m_k^* := \begin{cases} \max\{j \in \mathcal{M} : B_k + \hat{s}_j \leq (1 + \varepsilon)\delta_k\}, & \text{if this set is nonempty,} \\ 1, & \text{otherwise.} \end{cases}$$

Then the following hold.

1. If $\varepsilon \leq -1$, then the feasible set is empty for every arrival, and hence $m_k^* = 1$ for every arrival.
2. If $\varepsilon > -1$, define for each mode j

$$B_j^*(\delta_k, \varepsilon) := (1 + \varepsilon)\delta_k - \hat{s}_j.$$

Then

$$B_1^*(\delta_k, \varepsilon) \geq B_2^*(\delta_k, \varepsilon) \geq \dots \geq B_J^*(\delta_k, \varepsilon),$$

and, for arrivals with a nonempty feasible set, the selected mode is exactly tier-threshold based in estimated backlog:

$$m_k^* = j \iff B_k \leq B_j^*(\delta_k, \varepsilon) \text{ and } (j = J \text{ or } B_k > B_{j+1}^*(\delta_k, \varepsilon)).$$

If the feasible set is empty, the policy returns mode 1 by convention.

3. Equivalently, for fixed (R_k, Q_k) and $\varepsilon > -1$, define

$$\delta_j^*(R_k, Q_k; \varepsilon) := \frac{R_k + Q_k \hat{s}_{avg} + \hat{s}_j}{1 + \varepsilon}, \quad j = 1, \dots, J.$$

Then

$$\delta_1^*(R_k, Q_k; \varepsilon) \leq \delta_2^*(R_k, Q_k; \varepsilon) \leq \dots \leq \delta_J^*(R_k, Q_k; \varepsilon),$$

and, whenever at least one threshold is met, m_k^* is the largest index j such that

$$\delta_k \geq \delta_j^*(R_k, Q_k; \varepsilon).$$

If no threshold is met, the policy returns mode 1 by convention.

4. On the regime $\varepsilon > -1$, the selected mode is nonincreasing in R_k and Q_k , and nondecreasing in δ_k and ε .

For $J = 2$, with $(\rho_1, \rho_2) = (1 - \rho, \rho)$ and $(\hat{s}_1, \hat{s}_2) = (\hat{s}(\text{FAST}), \hat{s}(\text{SLOW}))$, this reduces to the two-mode FTC rule in Eqs. (9)–(10) and Theorem 4.1.

Proof. Because each $\hat{s}_j > 0$ and $\boldsymbol{\rho}$ is a probability vector,

$$\hat{s}_{avg} = \sum_{j=1}^J \rho_j \hat{s}_j > 0.$$

Hence, for any mode j , feasibility of choosing mode j is equivalent to the affine inequality

$$R_k + Q_k \hat{s}_{avg} + \hat{s}_j \leq (1 + \varepsilon) \delta_k,$$

or equivalently

$$B_k + \hat{s}_j \leq (1 + \varepsilon) \delta_k.$$

For item 1, if $\varepsilon \leq -1$, then $(1 + \varepsilon) \delta_k \leq 0$ because $\delta_k > 0$. Moreover, since $R_k \geq 0$, $Q_k \geq 0$, and $\hat{s}_{avg} > 0$,

$$B_k = R_k + Q_k \hat{s}_{avg} \geq 0,$$

and therefore

$$B_k + \hat{s}_j \geq \hat{s}_j > 0$$

for every j . Therefore no mode satisfies the feasibility inequality, so the feasible set is empty for every arrival. By definition, the policy then returns mode 1.

For item 2, assume $\varepsilon > -1$. Rearranging the feasibility condition for mode j gives

$$B_k \leq (1 + \varepsilon)\delta_k - \hat{s}_j = B_j^*(\delta_k, \varepsilon).$$

Since

$$\hat{s}_1 \leq \hat{s}_2 \leq \dots \leq \hat{s}_J,$$

it follows immediately that

$$B_1^*(\delta_k, \varepsilon) \geq B_2^*(\delta_k, \varepsilon) \geq \dots \geq B_J^*(\delta_k, \varepsilon).$$

Thus, if some mode j is feasible, then every faster mode $1, \dots, j-1$ is also feasible. Hence the feasible set is either empty or of the form

$$\{1, 2, \dots, j\}$$

for some j . Since the policy chooses the richest feasible mode, it selects exactly the largest feasible index, which proves the stated characterization for the nonempty-feasible-set case. If the feasible set is empty, the policy definition gives $m_k^* = 1$.

For item 3, because $\varepsilon > -1$, one has $1 + \varepsilon > 0$, so dividing the same feasibility inequality by $1 + \varepsilon$ is valid. Mode j is feasible if and only if

$$\delta_k \geq \frac{R_k + Q_k \hat{s}_{avg} + \hat{s}_j}{1 + \varepsilon} = \delta_j^*(R_k, Q_k; \varepsilon).$$

Since the $\hat{s}_j$ are ordered increasingly, the thresholds $\delta_j^*(R_k, Q_k; \varepsilon)$ are also ordered increasingly. Therefore, whenever at least one threshold is met, the selected mode is the largest index j whose TTL threshold is met. If no threshold is met, the policy definition gives $m_k^* = 1$.

For item 4, if R_k increases or Q_k increases, then

$$B_k = R_k + Q_k \hat{s}_{avg}$$

increases because $\hat{s}_{avg} > 0$, so the set of feasible modes can only shrink; therefore the selected index cannot increase. If δ_k increases or ε increases, then every threshold $B_j^*(\delta_k, \varepsilon) = (1 + \varepsilon)\delta_k - \hat{s}_j$ weakly increases, so the feasible set can only expand; therefore the selected index cannot decrease.

Finally, when $J = 2$, setting

$$(\hat{s}_1, \hat{s}_2) = (\hat{s}(\text{FAST}), \hat{s}(\text{SLOW})), \quad (\rho_1, \rho_2) = (1 - \rho, \rho),$$

yields

$$\hat{s}_{avg} = (1 - \rho)\hat{s}(\text{FAST}) + \rho\hat{s}(\text{SLOW}),$$

which is exactly Eq. (9), and the richest-feasible selection reduces to: choose SLOW if feasible, otherwise choose FAST, which is exactly the current two-mode FTC rule. $\square$

Theorem 4.2 shows that the threshold structure of FTC is not an artifact of the binary fast/slow abstraction. Once the available execution profiles are ordered by their service-time summaries, the same single-server deadline-feasibility argument yields a tiered threshold rule: tighter backlog or smaller TTL pushes the decision toward faster modes, while looser conditions admit progressively richer modes. Thus the current two-mode FTC should be viewed as the minimal special case of a broader finite-mode feasibility-gating family. This extension does not claim new optimality results; rather, it shows that the analytical identity proved in Theorem 4.1 persists when the mode menu is enlarged, with the queue-mix scalar replaced by a simplex-valued calibration parameter.

4.3. Oracle threshold benchmark and relationship to bang-bang control

The threshold form in Theorem 4.1 is a structural property of FTC itself. It is also useful to compare FTC with (i) a one-step oracle that optimizes expected on-time utility directly in slack space and (ii) the bang-bang feasibility controller F-BB. Unlike Theorem 4.1, the oracle result below is not implied by the queueing model alone: its single-crossing condition is an additional benchmark assumption on the mode-wise utility/CDF comparison.

Let $z = \delta - w$ denote residual slack when the actual waiting time is w . For $m \in \{\text{FAST}, \text{SLOW}\}$, let $\bar{u}_m = \mathbb{E}[U(m)]$ and

$$G_m(z) := \bar{u}_m F_m(z), \quad F_m(z) := \Pr(S(m) \leq z),$$

with $F_m(z) = 0$ for $z < 0$. Then $G_m(z)$ is the myopic expected on-time utility if mode m is chosen at slack z .

Theorem 4.3 (One-step oracle slack threshold). *Assume the difference*

$$D(z) := G_{\text{SLOW}}(z) - G_{\text{FAST}}(z) = \bar{u}_{\text{SLOW}} F_{\text{SLOW}}(z) - \bar{u}_{\text{FAST}} F_{\text{FAST}}(z)$$

has the single-crossing property: there exists $z^ \in \mathbb{R} \cup \{\pm\infty\}$ such that $D(z) < 0$ for $z < z^*$ and $D(z) \geq 0$ for $z \geq z^*$. Then the myopic utility-maximizing*

action is threshold-based in residual slack:

$$m^*(z) = \begin{cases} \text{FAST}, & z < z^*, \\ \text{SLOW}, & z \geq z^*. \end{cases}$$

Proof. For any fixed slack value z , the myopic expected on-time utility under mode m is exactly $G_m(z)$. Therefore the myopic controller chooses SLOW if and only if $G_{\text{SLOW}}(z) \geq G_{\text{FAST}}(z)$, equivalently if and only if $D(z) \geq 0$. Under the single-crossing assumption, the set $\{z : D(z) \geq 0\}$ is of the form $[z^*, \infty)$ (allowing the degenerate cases $z^* = \infty$ and $z^* = -\infty$). Hence the myopic utility-maximizing action is a threshold rule in residual slack. $\square$

Theorem 4.3 is an oracle benchmark rather than a direct implementation result. It shows that threshold behavior is also compatible with the underlying reward–deadline trade-off, not only with the algebra of FTC’s gate. FTC can therefore be interpreted as a lightweight plug-in approximation that replaces the full CDF comparison by a calibrated feasibility test based on backlog summaries.

Proposition 4.4 (F-BB as a special case of FTC). *Setting $\rho = 1$ and $\varepsilon = 0$ in FTC yields*

$$R_k + Q_k \hat{s}(\text{SLOW}) + \hat{s}(\text{SLOW}) \leq \delta_k,$$

which is exactly the F-BB rule in Eq. (13).

Proof. When $\rho = 1$, Eq. (9) gives $\hat{s}^{\text{avg}} = \hat{s}(\text{SLOW})$. Setting $\varepsilon = 0$ in Eq. (10) then yields

$$R_k + Q_k \hat{s}(\text{SLOW}) + \hat{s}(\text{SLOW}) \leq \delta_k,$$

which is Eq. (13). Therefore F-BB is recovered exactly as the special case $(\rho, \varepsilon) = (1, 0)$ of FTC. $\square$

Proposition 4.5 (Slow-admission region nesting and disagreement region). *Fix $\varepsilon = 0$. Let $a = \hat{s}^{\text{avg}}$ and $b = \hat{s}(\text{SLOW})$. Since $\hat{s}(\text{FAST}) \leq \hat{s}(\text{SLOW})$ in the fast/slow ordering and $a = \rho \hat{s}(\text{SLOW}) + (1 - \rho) \hat{s}(\text{FAST})$ is a convex combination of these two values, one has $a \leq b$. Define*

$$\mathcal{S}_{\text{FTC}} := \{(R, Q, \delta) : R + Qa + b \leq \delta\}, \quad \mathcal{S}_{\text{F-BB}} := \{(R, Q, \delta) : R + Qb + b \leq \delta\}.$$

Then

$$\mathcal{S}_{\text{F-BB}} \subseteq \mathcal{S}_{\text{FTC}}.$$

Moreover, the states in which FTC chooses SLOW and F-BB chooses FAST are exactly

$$\mathcal{D} := \{(R, Q, \delta) : R + Qa + b \leq \delta < R + Qb + b\}.$$

Proof. Because $a \leq b$, one has $R + Qa + b \leq R + Qb + b$ for every (R, Q) . Therefore any state that satisfies the F-BB feasibility inequality also satisfies the FTC feasibility inequality at $\varepsilon = 0$, which proves the set inclusion. The disagreement region is precisely the set of states in which FTC's inequality holds while F-BB's inequality does not, giving the stated characterization of $\mathcal{D}$. $\square$

Corollary 4.6 (State-dependent crossover under negative margins). *With the notation of Proposition 4.5, FTC requires a larger TTL threshold than F-BB at state (R, Q) if and only if*

$$\frac{R + Qa + b}{1 + \varepsilon} > R + Qb + b,$$

equivalently,

$$\varepsilon < -\frac{Q(b - a)}{R + Qb + b}.$$

Thus negative ε provides an explicit mechanism by which FTC can move from a more aggressive feasibility gate to a more conservative one than F-BB.

Proof. At state (R, Q) , FTC requires a TTL of at least $(R + Qa + b)/(1 + \varepsilon)$, whereas F-BB requires a TTL of at least $R + Qb + b$. FTC is more conservative exactly when the former threshold exceeds the latter. Since $R + Qb + b > 0$, rearranging the inequality yields the stated condition on ε . $\square$

4.4. Backlog-estimator refinement and compression error

The queue-mix estimator in Eq. (8) is intentionally lightweight because it needs only (R_k, Q_k) and a scalar ρ . When the scheduler tracks the already assigned modes of queued jobs, a more detailed benchmark is available. Let $n_f(k)$ and $n_s(k)$ denote the numbers of queued FAST and SLOW jobs at the arrival of request k , so that $n_f(k) + n_s(k) = Q_k$. Define the mode-aware benchmark

$$\widehat{W}_k^{\text{mode}} := R_k + n_f(k) \hat{s}(\text{FAST}) + n_s(k) \hat{s}(\text{SLOW}).$$

Also define the original FTC estimator as

$$\widehat{W}_k^\rho := R_k + Q_k [\rho \hat{s}(\text{SLOW}) + (1 - \rho) \hat{s}(\text{FAST})].$$

Let

$$\widehat{\rho}_k := \begin{cases} \frac{n_s(k)}{Q_k}, & Q_k > 0, \\ 0, & Q_k = 0. \end{cases}$$

Proposition 4.7 (Compression error identity for the queue-mix estimator).
For every arrival k ,

$$\widehat{W}_k^\rho - \widehat{W}_k^{\text{mode}} = Q_k (\rho - \widehat{\rho}_k) (\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})).$$

Consequently,

$$|\widehat{W}_k^\rho - \widehat{W}_k^{\text{mode}}| \leq Q_k |\rho - \widehat{\rho}_k| (\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})).$$

Proof. Because $n_f(k) = Q_k - n_s(k)$, one has

$$\begin{aligned} \widehat{W}_k^{\text{mode}} &= R_k + (Q_k - n_s(k)) \hat{s}(\text{FAST}) + n_s(k) \hat{s}(\text{SLOW}) \\ &= R_k + Q_k \hat{s}(\text{FAST}) + n_s(k) (\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})). \end{aligned}$$

Similarly,

$$\widehat{W}_k^\rho = R_k + Q_k \hat{s}(\text{FAST}) + Q_k \rho (\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})).$$

Subtracting the two identities gives

$$\widehat{W}_k^\rho - \widehat{W}_k^{\text{mode}} = (Q_k \rho - n_s(k)) (\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})) = Q_k (\rho - \widehat{\rho}_k) (\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})),$$

where the last step is immediate when $Q_k = 0$ as both sides vanish. Taking absolute values yields the bound. $\square$

Proposition 4.7 clarifies when the lightweight estimator is reliable: the compression error grows linearly with queue length, with the latency gap between the two modes, and with the mismatch between the calibrated ρ and the realized slow-job fraction already sitting in the queue. This also suggests a concrete diagnostic for the empirical section: report the realized queue-mix distribution and compare FTC to a mode-aware variant whenever the scheduler can track queued mode labels.

Algorithm 1 FTC (Firm-Deadline Tool Control) at request arrival (defaults: $\rho = 0.55$, $\varepsilon = 0$).

```

1: Input: observable state  $\mathbf{X}_k = (R_k, Q_k)$ ; TTL budget  $\delta_k$ ; mode mean
   service times  $\hat{s}(\text{SLOW})$ ,  $\hat{s}(\text{FAST})$ ; parameters  $\rho \in [0, 1]$ ,  $\varepsilon \in \mathbf{R}$ 
2: Output: selected mode in  $\{\text{SLOW}, \text{FAST}\}$ 
3:  $\hat{s}^{\text{avg}} \leftarrow \rho \hat{s}(\text{SLOW}) + (1 - \rho) \hat{s}(\text{FAST})$ 
4:  $\widehat{W}_k \leftarrow R_k + Q_k \hat{s}^{\text{avg}}$ 
5: if  $\widehat{W}_k + \hat{s}(\text{SLOW}) \leq (1 + \varepsilon) \delta_k$  then
6:   return SLOW
7: else
8:   return FAST
9: end if

```

4.5. Algorithm

Algorithm 1 summarizes the same arrival-epoch decision rule. The theorem above shows that this algorithm implements a monotone threshold policy rather than a purely procedural heuristic. The computation is $O(1)$ per arrival given mode summaries $\hat{s}(\cdot)$.

4.6. Parameterization and operating point

Selecting ρ . In our evaluation, we select ρ via a lightweight grid search over $\rho \in [0, 1]$ on a held-out configuration and then fix it for all experiments. Intuitively, smaller ρ makes the backlog estimate more optimistic (assuming more queued jobs will run in FAST), while larger ρ makes it more conservative. Note that $\rho = 0.55$ is fixed for the remainder of the document unless stated otherwise.

Selecting ε . The time margin ε controls the strictness of the feasibility test in (7). The default $\varepsilon = 0$ corresponds to a strict predicted-feasibility rule. Sweeping $\varepsilon \in \mathbf{R}$ relaxes or tightens the gate and traces a utility–deadline-miss operating frontier. The practically relevant regime is $\varepsilon > -1$; for $\varepsilon \leq -1$, FTC degenerates to ALWAYS-FAST by Theorem 4.1.

FTC* operating point. When we write FTC* in results, it denotes the proposed FTC policy instantiated with its default tuned parameters, namely $\rho = 0.55$ and $\varepsilon = 0$, as determined via the grid-search procedure above.

Mode service-time summaries. FTC requires mode-wise service-time summaries $\hat{s}(\text{FAST})$ and $\hat{s}(\text{SLOW})$. In our experiments, these are computed from empirical latency traces; in deployment, they may be maintained as

rolling averages or quantiles over recent observations. The policy itself is not specific to one model identity: its inputs are queue state, deadline budget, and empirical mode-wise latency summaries. Any tool-augmented model/router stack that induces a measurable Tactical/Strategic latency contrast can therefore be calibrated into the same framework by replacing these summaries and replay distributions with traces collected from that stack.

4.7. Context-Aware Dynamic Tool Resolution (CADTR)

FTC treats every task identically: the feasibility gate uses the same ε regardless of task semantics. In mission-critical deployments, however, certain events carry qualitatively higher stakes—a critical threat detection, a time-sensitive safety alert, or a high-priority autonomous action. Missing the deadline on such a task is not merely a performance degradation but a *catastrophic failure*.

CADTR extends FTC with a minimal but consequential mechanism: *event-driven dynamic risk adaptation*. When a critical-priority event arrives, CADTR applies a context-dependent additive shift $\Delta_\varepsilon^{\text{crit}} < 0$ to the base ε , tightening the feasibility budget and biasing the policy toward the Tactical tool. For standard events, CADTR behaves identically to FTC.

Formal mechanism. For request k with priority class $p_k \in \{\text{critical}, \text{standard}\}$, CADTR selects the effective margin as

$$\varepsilon_k^{\text{eff}} = \begin{cases} \varepsilon + \Delta_\varepsilon^{\text{crit}}, & p_k = \text{critical}, \\ \varepsilon, & p_k = \text{standard}, \end{cases} \quad (11)$$

and then applies the standard FTC feasibility test (Eq. (7)) with $\varepsilon_k^{\text{eff}}$ replacing ε :

$$\widehat{W}_k + \hat{s}(\text{SLOW}) \leq (1 + \varepsilon_k^{\text{eff}}) \delta_k. \quad (12)$$

Interpretation. A negative $\Delta_\varepsilon^{\text{crit}}$ makes the feasibility gate *harder to pass* for critical tasks: CADTR proactively sacrifices the Strategic intelligence bonus to protect the critical task’s physical survival. This is a dynamic, event-level decision—not a static policy parameter—and it creates an asymmetric safety contract: critical tasks are shielded from congestion-induced deadline failure while standard tasks continue to use the full Strategic capability when feasible.

In our experiments, we use $\Delta_\varepsilon^{\text{crit}} = -0.20$, which tightens the feasibility window by 20% of the deadline budget for critical events. Algorithm 2 summarizes the full CADTR decision procedure.

Algorithm 2 CADTR (Context-Aware Dynamic Tool Resolution) at request arrival.

```

1: Input: state  $\mathbf{X}_k = (R_k, Q_k)$ ; deadline budget  $\delta_k$ ; priority  $p_k \in \{\text{crit}, \text{std}\}$ ;
   mode service times  $\hat{s}(\text{SLOW}), \hat{s}(\text{FAST})$ ; base margin  $\varepsilon$ ; critical shift  $\Delta_\varepsilon^{\text{crit}}$ ;
   queue-mix  $\rho$ 
2: Output: selected mode in  $\{\text{SLOW}, \text{FAST}\}$ 
3: if  $p_k = \text{critical}$  then
4:    $\varepsilon_k^{\text{eff}} \leftarrow \varepsilon + \Delta_\varepsilon^{\text{crit}}$ 
5: else
6:    $\varepsilon_k^{\text{eff}} \leftarrow \varepsilon$ 
7: end if
8:  $\hat{s}^{\text{avg}} \leftarrow \rho \hat{s}(\text{SLOW}) + (1 - \rho) \hat{s}(\text{FAST})$ 
9:  $\widehat{W}_k \leftarrow R_k + Q_k \hat{s}^{\text{avg}}$ 
10: if  $\widehat{W}_k + \hat{s}(\text{SLOW}) \leq (1 + \varepsilon_k^{\text{eff}}) \delta_k$  then
11:   return SLOW {Strategic tool}
12: else
13:   return FAST {Tactical tool}
14: end if

```

Structural inheritance. Because CADTR is FTC with a state-dependent ε substitution, the monotonicity structure of Theorem 4.1 is inherited: within each priority class, CADTR remains a monotone threshold policy in (R_k, Q_k, δ_k) . The multi-mode generalization (Theorem 4.2) also extends directly by substituting $\varepsilon_k^{\text{eff}}$ into the tier-threshold rule.

5. Baseline Policies

We evaluate CADTR against four baseline orchestration policies, deliberately designed to span a hierarchy of information access—from basic feasibility gating (F-BB) to full single-task queue-wait knowledge (Mode-Aware Oracle) and optimal expected-utility maximization (Myopic CDF Oracle). All policies decide at the arrival epoch and share the same queueing dynamics, service-time/utility models, and experimental conditions. All hyperparameters are selected via grid search on a held-out configuration and then fixed for the reported experiments.

5.1. *FTC** (tuned FTC baseline)

FTC* denotes the base FTC policy (Algorithm 1) instantiated with its default tuned parameters ($\rho = 0.55$, $\varepsilon = 0$). FTC* treats all tasks identically—it has no priority-dependent adaptation. Comparing CADTR to FTC* isolates the contribution of the context-dependent ε -shift.

5.2. *TTL-feasibility bang-bang (F-BB)*

F-BB is a conservative feasibility baseline that uses the all-SLOW workload estimate

$$\widetilde{W}_k := R_k + Q_k \hat{s}(\text{SLOW})$$

and selects SLOW if and only if

$$\widetilde{W}_k + \hat{s}(\text{SLOW}) \leq \delta_k, \quad (13)$$

otherwise it selects FAST. F-BB uses residual work, queue length, and deadline budget, but assumes that every queued job consumes the full Strategic mean service time and exposes neither the queue-mix parameter ρ nor the operating-point parameter ε .

5.3. *Mode-Aware Oracle*

The Mode-Aware Oracle computes the exact predicted waiting time for each arriving task by inspecting the *actual assigned mode* (Tactical or Strategic) of every task already in the queue, rather than approximating via the queue-mix parameter ρ . For request k with Q_k queued tasks, the waiting time is computed as

$$\widehat{W}_k^{\text{MA}} = R_k + \sum_{j=1}^{Q_k} \hat{s}(m_j), \quad (14)$$

where m_j is the mode already assigned to the j -th queued task. The oracle then applies the same FTC feasibility test using this exact backlog estimate. This baseline controls for the approximation error introduced by the queue-mix parameter ρ : if FTC’s aggregate backlog estimate is materially inaccurate, the Mode-Aware Oracle should outperform it. It has *strictly more information* than CADTR, since it knows every queued task’s mode assignment.

Table 1: Information requirements and decision logic of evaluated policies.

Policy	Queue state	Per-task modes	Deadline	Priority
F-BB	(R_k, Q_k)	No	Yes	No
FTC*	(R_k, Q_k)	No	Yes	No
Mode-Aware Oracle	(R_k, Q_k)	Yes	Yes	No
CDF Oracle	(R_k, Q_k)	Yes	Yes	No
CADTR	(R_k, Q_k)	No	Yes	Yes

5.4. Myopic CDF Oracle

The Myopic CDF Oracle implements the single-task expected-utility maximizer derived from the theoretical analysis (related to Theorem 4.2). For each arriving task, it selects the mode m^* that maximizes the expected on-time utility:

$$m_k^* = \arg \max_{m \in \{\text{FAST}, \text{SLOW}\}} \hat{u}(m) \cdot \Phi \left(\frac{\delta_k - \widehat{W}_k^{\text{MA}} - \hat{s}(m)}{\hat{\sigma}(m)} \right), \quad (15)$$

where $\hat{u}(m)$ is the expected mode utility, $\widehat{W}_k^{\text{MA}}$ is the Mode-Aware Oracle’s exact waiting-time estimate (Eq. 14), $\hat{s}(m)$ and $\hat{\sigma}(m)$ are the empirical mean and standard deviation of service time under mode m , and $\Phi(\cdot)$ is the standard normal CDF.

This oracle combines the Mode-Aware Oracle’s exact queue knowledge with a probabilistic completion model, and it greedily maximizes per-task expected utility. It is the strongest conceivable single-task baseline: it uses more information (exact per-task modes) and a more sophisticated decision rule (expected-utility optimization) than CADTR. When this oracle fails under load, it reveals that myopic per-task optimization cannot protect critical tasks—a qualitative insight that motivates CADTR’s dynamic risk adaptation.

5.5. Summary of information requirements

Table 1 highlights a key asymmetry: both oracles access strictly more information than CADTR (exact per-task mode assignments), yet CADTR accesses a different *kind* of information (event priority). As the results in Section 7 demonstrate, the qualitative semantic signal (“is this task critical?”) proves more valuable for protecting critical tasks than the quantitative informational advantage of exact queue-wait computation.

6. Experimental Setup

We evaluate FTC and baselines using discrete-event simulation driven by empirically measured end-to-end latencies for tool-augmented LLM inference. This section specifies the simulator, trace sources, deadline (TTL) regimes, utility generation, hyperparameter selection, and reported metrics.

6.1. Discrete-event simulator

We simulate a non-preemptive single-server queue with an infinite waiting room and firm deadlines as defined in Section 3. Arrivals follow a Poisson process with rate λ . Upon each arrival, the policy selects a mode $m_k \in \{\text{FAST}, \text{SLOW}\}$. Service time $S_k(m_k)$ and success utility $U_k(m_k)$ are sampled as described below. A request yields realized utility $\tilde{U}_k = U_k(m_k)\mathbf{1}\{c_k \leq d_k\}$ (Eq. (4)). We report steady-state averages after a warm-up period; all point estimates are averaged across multiple random seeds (see Section 6.8).

6.2. Empirical service-time traces

For each mode m , we obtain empirical end-to-end latency samples from a dedicated trace-generation harness and implement the service-time distribution F_m by trace replay (sampling with replacement). Each sample measures the full execution time of the corresponding pipeline, including model generation and any tool overhead present in SLOW. This keeps the simulator nonparametric at the service-time layer and preserves the heavy tails, multimodality, and mode separation observed in the measured traces.

Counterfactual paired measurements. A naive trace-collection protocol can confound prompt difficulty with mode choice: if a router tends to send harder prompts to the tool-rich pipeline, then mode-wise latency pools mix the effect of the prompt with the effect of the mode. To reduce this selection bias, we additionally collect counterfactual pairs by executing the same prompt under both modes. The resulting paired samples condition on the prompt and isolate the marginal latency effect of enabling the slow tool-rich pipeline. These pairs are used both to estimate mode-wise service-time summaries and to support controlled offline policy comparisons under a common prompt mix. Unless otherwise noted, the simulation uses the mode-wise empirical distributions derived from these paired measurements.

Table 2: Deadline (TTL) regimes used in experiments: $\delta_k \sim \mathcal{N}(\mu_\delta, \sigma_\delta^2)$

Regime	μ_δ (s)	σ_δ (s)
Regime I	28.0	9.0
Regime II	35.0	10.0
Regime III	45.0	12.0
Regime IV	60.0	15.0

6.3. Deadline (TTL) regimes

Each request k has a TTL budget δ_k and absolute deadline $d_k = a_k + \delta_k$. Our primary experiments model TTL heterogeneity via a bounded (clipped) normal distribution:

$$\delta_k \sim \text{clip}(\mathcal{N}(\mu_\delta, \sigma_\delta^2), \delta_{\min}, \delta_{\max}), \quad (16)$$

where $(\mu_\delta, \sigma_\delta)$ are set per regime as given in Table 2 and $(\delta_{\min}, \delta_{\max})$ are $(0.05, \infty)$ respectively for all regimes. We report the regime parameters in Table 2.

Rationale and static-TTL sensitivity. Although many interactive systems enforce a nominally fixed timeout, effective time budgets can vary due to tiering, upstream latency, retry budgets, or policy-driven SLO adjustments. This reflects scenarios where user-facing timeouts vary around a mean with some variation. The clipped-normal model provides a controlled way to introduce deadline heterogeneity while holding mean budgets comparable across regimes. To verify that conclusions do not depend on TTL randomness, we also evaluate a static TTL setting:

$$\delta_k \equiv \mu_\delta, \quad (17)$$

and report consistent qualitative ordering of policies under both models.

6.4. Utility generation

The utility model instantiates the multi-resolution framework from Section 3.4. For each request, the mode-specific success utility is drawn from a peaked distribution centered on the physically motivated values:

$$\begin{aligned} U_k(\text{FAST}) &\sim \mathcal{N}(u_{\text{base}}, \sigma_u^2), & u_{\text{base}} &= 1.0, \\ U_k(\text{SLOW}) &\sim \mathcal{N}(u_{\text{base}} + u_{\text{intel}}, \sigma_u^2), & u_{\text{intel}} &= 0.5, \end{aligned}$$

with $\sigma_u = 0.02$ providing minimal organic noise. All values are clipped to $[0, \infty)$. The firm-deadline rule $\tilde{U}_k = U_k(m_k)\mathbf{1}\{c_k \leq d_k\}$ (Eq. (4)) applies, so missed deadlines yield $\tilde{U}_k = 0$ (catastrophic failure).

Critical-task utility scaling. For the two-class experiments (Section 3.6), critical tasks receive a $1.2\times$ utility multiplier, reflecting the higher operational stakes of mission-critical events. Standard tasks use the base utility values above.

6.5. Policy parameters and hyperparameter selection

FTC and CADTR defaults. Unless explicitly varied, FTC uses $\varepsilon = 0$ and $\rho = 0.55$ (selected via grid search on a held-out configuration and fixed for all experiments). CADTR inherits these FTC parameters and additionally uses the critical-event ε -shift $\Delta_\varepsilon^{\text{crit}} = -0.20$.

Baseline parameters. The F-BB baseline requires no tunable parameters beyond the mode service-time summaries. Both oracle baselines (Mode-Aware and CDF Oracle) use the same empirical service-time summaries as FTC but compute exact per-task waiting times from actual queue mode assignments.

Lambda sweep. Our primary experiment sweeps the arrival rate λ across nine values from 0.03 to 0.19 req/s, with a critical-task fraction of 25%. The TTL regime uses $(\mu_\delta, \sigma_\delta) = (35, 10)$ with clipping to $[15, 65]$. Results are averaged over 5 replications (seed base 42). Each replication simulates 1500 events with a 200-event warm-up period.

6.6. Trace-generation system and LLM configuration

Empirical service-time traces are obtained from a dedicated trace-generation harness that executes a tool-routing agent on a fixed local compute environment. The generator uses an Ollama backend and three foundation-model backends: Llama 3.1, Qwen 2.5 7B Instruct, and Mistral 7B Instruct. For each prompt and for each model backend, we record counterfactual paired service times by executing both the FAST and SLOW instructions, regardless of the router’s chosen tool, producing paired $(S^{\text{fast}}, S^{\text{slow}})$ samples for the same request. The resulting dataset contains per-prompt metadata, router outputs, latency measurements for both modes, the model-backend identifier, and additional response attributes such as response length.

For the main queueing experiments, the paired traces from all three model backends are pooled and shuffled before empirical replay. Thus, unless stated

otherwise, simulator results should be interpreted as policy performance on the combined multi-model workload rather than on a backend-specific subset.

Serving stack and dataset structure. The trace-generation stack consists of four layers: (i) a prompt set submitted to a local tool-routing agent, (ii) an Ollama-served foundation-model backend (Llama 3.1, Qwen 2.5 7B Instruct, or Mistral 7B Instruct), (iii) two execution profiles exposed to the agent, namely a lighter fast profile that requests a shorter answer and a richer slow profile that requests a longer answer, and (iv) a logging layer that records prompt-level metadata, router decisions, error indicators, response lengths, the model-backend identifier, and the paired end-to-end latencies for both modes on the same prompt. The prompt set is deliberately mixed: it includes short factual or arithmetic requests and longer prompts that require explanation, comparison, planning, or multi-step reasoning; representative examples are listed in Appendix A. The important point is that the same underlying prompt is executed in both modes for each model backend, so each dataset row represents a matched fast/slow pair for a fixed query and model identity.

Why the paired design matters. The paired counterfactual protocol is important for causal hygiene in the latency data. Without pairing, mode-specific trace pools can inherit the router’s prompt-selection effects; for example, difficult prompts may be overrepresented in the tool-rich mode. Executing both modes on the same prompt conditions on prompt content and therefore reduces selection bias in the estimated fast/slow latency gap. For the queueing study, this gives a cleaner estimate of the marginal service-time impact of enabling the richer tool profile and yields a more defensible offline replay workload.

Mode characterization. The two modes differ not only in mean latency, but also in response style and intended task type. **fast_lookup** is designed for short factual answers and single-step lookups; in the trace database (excluding errored rows; $n = 1332$), response length in characters is typically small (mean ≈ 173 , median ≈ 156 , $p_{10} \approx 30$, $p_{90} \approx 333$, min = 2, max = 1736). **deep_reasoner** is designed for longer, explanation-oriented, or multi-step responses; in the same trace database ($n = 1332$), responses are substantially longer (mean ≈ 3837 , median ≈ 3761 , $p_{10} \approx 2398$, $p_{90} \approx 5343$, min = 1410, max = 7618). This distinction is exactly the application-level phenomenon that motivates the FAST/SLOW abstraction in the queueing model.

What matters for the queueing study. For the queueing results, the material properties of the LLM setup are the empirical mode-wise service-time distributions, their within-prompt fast/slow contrast, and the fact that the same hardware and software stack is used throughout trace collection. These properties determine the calibration of $\hat{s}(\text{FAST})$, $\hat{s}(\text{SLOW})$, and the replay distributions F_m . By contrast, the exact wording style of the responses, the internal architecture of the local model, or the semantic quality of any one answer are not first-order inputs to the queueing analysis; they matter only insofar as they induce different latency profiles across the two modes.

Computation environment. Trace collection was tested on Windows 11 (64-bit) with an AMD Ryzen 7 5800H CPU, an RTX 3080 Laptop GPU (8GB), and 31.4 GiB RAM, using Python 3.9.13. These specifications are reported to contextualize absolute latencies. The policy comparison itself is based on relative performance under a common trace distribution rather than on claims about one absolute hardware point.

External validity. The trace source is still intentionally controlled: it uses a fixed local hardware environment, a two-tool router schema, and three local model backends (Llama 3.1, Qwen 2.5 7B Instruct, and Mistral 7B Instruct). We therefore do not claim that the absolute latency values will transfer unchanged to every tool-augmented LLM deployment. At the same time, the queueing conclusions are not tied to one model identity. FTC does not use model-specific internals, token-level features, or prompt semantics; it uses backlog, TTL, and calibrated mode-wise latency summaries. Across all three measured backends, the same qualitative fast/slow separation remains visible, supporting the claim that the feasibility-gating mechanism transfers at the methodology level beyond a single model family.

6.7. Metrics

We report: (i) *deadline-miss rate (DMR)* $\Pr[c_k > d_k]$, (ii) *mean on-time utility* $\mathbb{E}[\tilde{U}_k]$, and (iii) latency summary statistics (e.g., p95 end-to-end latency) where appropriate. For strict-priority experiments, we additionally report per-class metrics and the resulting class-wise utility–timeliness trade-offs.

6.8. Reproducibility

For each configuration (λ , TTL regime, policy), we run multiple independent simulation replications with different random seeds and report averages

across seeds after a warm-up period. All simulator logic, trace processing, and experiment scripts are available in an anonymized repository provided for blinded review.

7. Results

We evaluate CADTR against FTC*, F-BB, Mode-Aware Oracle, and Myopic CDF Oracle using the trace-driven simulator described in Section 6. The primary experiment sweeps arrival rate λ from 0.03 to 0.19 req/s with 25% critical tasks and deadline regime $(\mu_\delta, \sigma_\delta) = (35, 10)$. All results are averaged over 5 replications. We report per-class and overall metrics: miss rate (DMR), mean realized utility, and Strategic tool (slow-mode) usage fraction.

7.1. Critical task miss rate: *The Hero Graph*

Figure 1 shows the central finding: CADTR maintains a practically flat $\sim 3.5\%$ critical-task miss rate across the full load sweep, while the oracle baselines—which have *strictly more information*—degrade catastrophically as load increases. At $\lambda = 0.11$, the CDF Oracle drops 14.4% of critical tasks and the Mode-Aware Oracle drops 11.3%, compared to CADTR’s 3.5%. At $\lambda = 0.19$, the oracles still miss $\sim 11\text{--}14\%$ of critical tasks while CADTR holds at 3.5%.

This result is not explained by informational advantage: CADTR uses *less* queue-state information than either oracle. The difference is CADTR’s event-driven ε -shift, which provides *semantic awareness*—the ability to distinguish “this is a critical event that must not fail” from “this is a routine task.” The oracles, despite their mathematical sophistication, are blind to event semantics and treat all tasks identically.

7.2. The shield mechanism: *Dynamic Strategic tool throttling*

How does CADTR protect critical tasks? Figure 2 reveals the mechanism: CADTR dynamically throttles Strategic tool usage on critical tasks from 41% at low load ($\lambda = 0.03$) down to 5% at high load ($\lambda = 0.19$). In contrast, the oracle baselines maintain $\sim 9\%$ Strategic usage at high load—insufficient throttling because they cannot distinguish critical from standard tasks.

This adaptive behavior is CADTR’s “shield”: when the system detects congestion, it proactively sacrifices the intelligence bonus on critical tasks to guarantee their physical survival. The ε -shift creates a *safe corridor* in the feasibility space where critical tasks are preferentially routed to the Tactical (fast) tool, reducing their exposure to congestion-induced deadline failure.

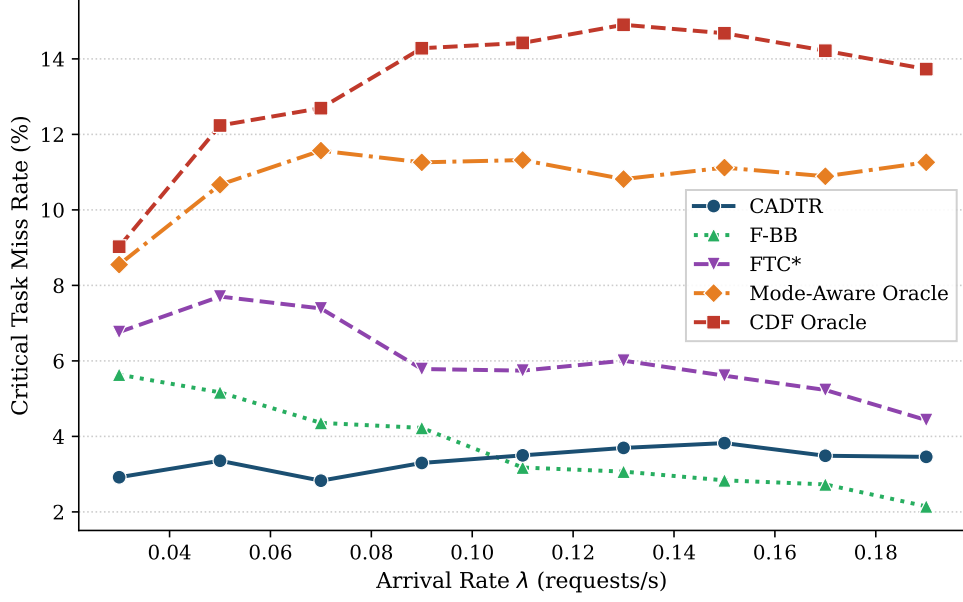


Figure 1: Critical task miss rate vs. arrival rate λ . CADTR maintains a flat $\sim 3.5\%$ miss rate while both oracle baselines degrade to 11–14% under load.

7.3. Overall system utility under load

Figure 3 shows that CADTR’s critical-task protection does not come at the expense of system-level utility. Across the load sweep, CADTR and F-BB maintain the highest overall utility, while both oracle baselines suffer utility degradation from their elevated miss rates—a direct consequence of their inability to shield critical tasks.

7.4. Overall deadline miss rate

Figure 4 presents the aggregate (critical + standard) deadline miss rate. F-BB achieves the lowest overall miss rate due to its ultra-conservative all-SLOW backlog assumption, but this comes at no informational cost advantage. CADTR achieves the second-lowest overall miss rate, outperforming both oracle baselines across the sweep. The CDF Oracle, despite being the most mathematically sophisticated, has the worst aggregate miss rate at high load—confirming that myopic per-task expected-utility optimization fails under congestion.

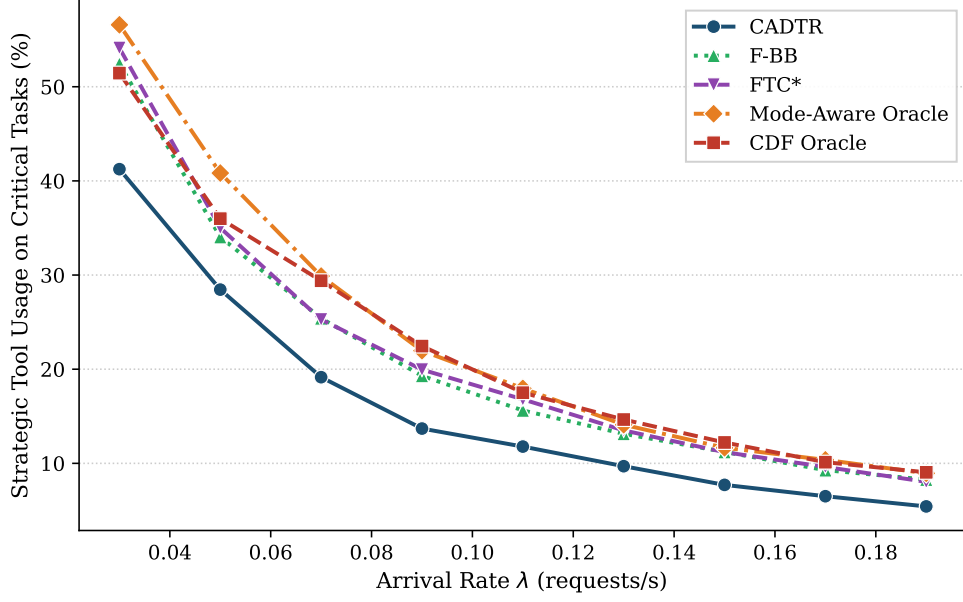


Figure 2: Strategic tool usage on critical tasks vs. λ . CADTR dynamically throttles from 41% to 5%, while oracles remain at $\sim 9\%$ at high load.

7.5. Quantitative summary at selected load points

Table 3 provides a detailed numerical comparison at three representative load levels: low ($\lambda = 0.03$), medium ($\lambda = 0.11$), and high ($\lambda = 0.19$).

Several observations emerge from Table 3:

- **CADTR dominates on critical miss rate:** CADTR achieves the lowest critical miss rate at low and medium load, and near-lowest at high load (3.5% vs. F-BB’s 2.1%), while maintaining substantially higher utility than F-BB at low load.
- **Oracle failure is systematic:** Both oracles consistently perform worst on critical tasks, despite having strictly more information than CADTR. This confirms that the failure is structural, not due to parameter miscalibration.
- **Dynamic throttling intensity scales with load:** CADTR’s Strategic usage on critical tasks drops from 41% to 5% as λ increases, demonstrating load-responsive behavior.

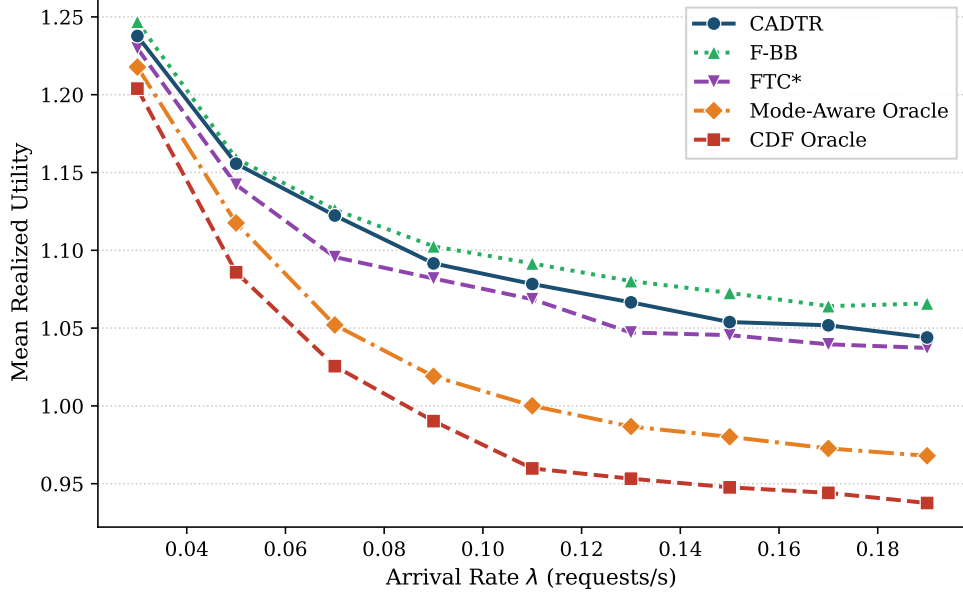


Figure 3: Overall mean realized utility vs. λ . CADTR maintains competitive system utility while providing superior critical-task protection.

- **F-BB trades utility for safety:** F-BB achieves the lowest *overall* miss rate at high load, but at the cost of over-conservative Tactical bias and lower utility at low load. CADTR achieves a better balance.

7.6. Empirical service-time distributions

Figure 5 plots the empirical ECDF of measured service latencies for the Tactical (*fast*) and Strategic (*slow*) pipelines across the three model back-ends: Llama 3.1, Qwen 2.5 7B Instruct, and Mistral 7B Instruct. The same qualitative pattern holds in each case: the Strategic profile is shifted to the right and is typically more variable than the corresponding Tactical profile. This cross-model consistency supports the multi-resolution tool abstraction and helps explain why backlog-aware admission control can achieve large reductions in deadline misses without collapsing to an always-Tactical regime.

8. Discussion

This section analyzes why CADTR’s minimal mechanism outperforms mathematically sophisticated oracles, explains the robustness of feasibility

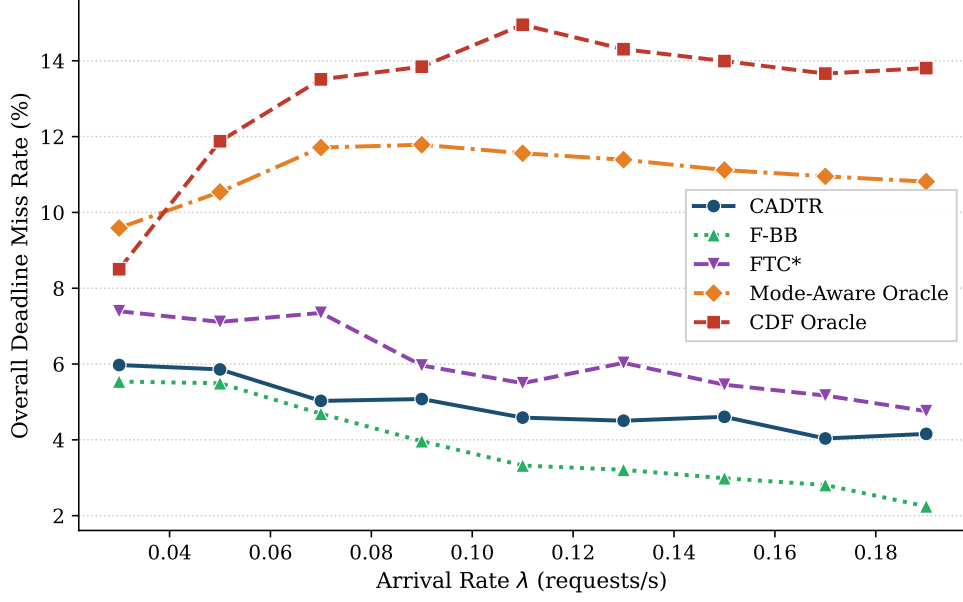


Figure 4: Overall deadline miss rate vs. λ across all task classes.

gating, and discusses limitations and future directions.

8.1. Why oracles fail: The limits of myopic optimization

The most striking result is that both oracle baselines—which possess strictly more information than CADTR—fail catastrophically on critical tasks under load. This is not a calibration artifact; it is a fundamental limitation of *semantic blindness*.

The Mode-Aware Oracle computes exact per-task waiting times by inspecting every queued task’s assigned mode, eliminating the approximation error of the ρ -based backlog estimator. The CDF Oracle goes further, maximizing expected on-time utility via Gaussian CDF approximation. Yet both treat critical and standard tasks identically. Under congestion, this symmetry is fatal: a mathematically optimal per-task decision that is blind to task semantics will accept the same level of risk for a routine status update and a mission-critical threat alert.

CADTR breaks this symmetry with a single additive ε -shift. The mechanism is trivial in implementation (one conditional branch in Algorithm 2), yet it creates a qualitatively different outcome: critical tasks are proactively routed to the Tactical (fast) tool under congestion, sacrificing the intelligence

Table 3: Performance at representative load points ($\lambda \in \{0.03, 0.11, 0.19\}$). Critical miss rate (Crit. MR), overall miss rate (Ovr. MR), overall utility (Ovr. Util.), and Strategic usage fraction on critical tasks (Crit. Strat.%). Best critical MR in **bold**.

λ	Policy	Crit. MR (%)	Ovr. MR (%)	Ovr. Util.	Crit. Strat. (%)
0.03	CADTR	2.9	6.0	1.238	41.2
	F-BB	5.6	5.5	1.247	52.5
	FTC*	6.8	7.4	1.230	54.1
	Mode-Aware	8.6	9.6	1.218	56.6
	CDF Oracle	9.0	8.5	1.204	51.4
0.11	CADTR	3.5	4.6	1.078	11.8
	F-BB	3.2	3.3	1.092	15.6
	FTC*	5.7	5.5	1.069	16.8
	Mode-Aware	11.3	11.6	1.000	18.0
	CDF Oracle	14.4	15.0	0.960	17.5
0.19	CADTR	3.5	4.2	1.044	5.4
	F-BB	2.1	2.2	1.066	8.3
	FTC*	4.4	4.8	1.037	8.0
	Mode-Aware	11.3	10.8	0.968	8.9
	CDF Oracle	13.7	13.8	0.938	9.0

bonus to guarantee survival. The lesson is that *semantic awareness beats mathematical sophistication* when the cost of failure is asymmetric across task classes.

8.2. Dynamic tool throttling as a safety mechanism

CADTR’s load-adaptive Strategic tool throttling (Figure 2) functions as an automatic safety mechanism. At low load, critical tasks use the Strategic tool 41% of the time, benefiting from the full intelligence bonus. As congestion grows, CADTR progressively restricts Strategic usage to 5%, concentrating the system’s deadline-feasibility budget on survival. This behavior emerges naturally from the ε -shift interacting with the monotone threshold structure (Theorem 4.1): a tighter ε contracts the Strategic-tool admission region in exactly the cases where the backlog threatens deadline feasibility.

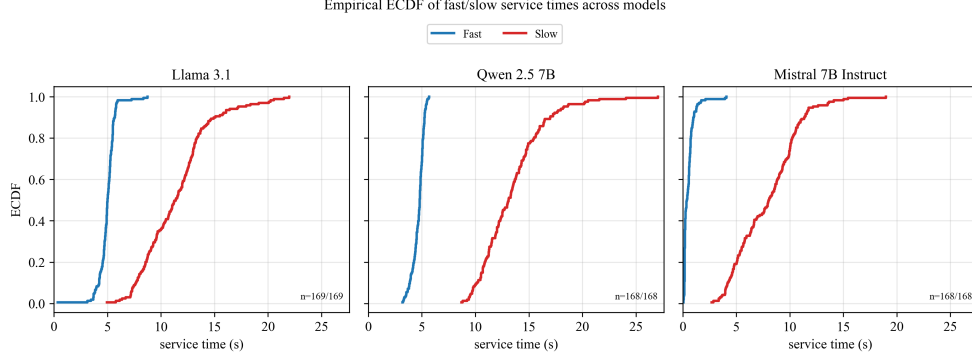


Figure 5: Empirical ECDF of service times for Tactical (FAST) and Strategic (SLOW) pipelines across three counterfactually measured model backends. In each panel, the Strategic pipeline is stochastically slower, supporting the multi-resolution tool abstraction.

8.3. Why feasibility gating is robust under congestion

Feasibility gating couples mode selection to the same quantity that defines timeliness: the remaining deadline budget δ_k . When backlog grows, the waiting-time estimate $\widehat{W}_k$ increases, causing more arrivals to be assigned to the Tactical tool, which reduces the induced mean service time and stabilizes queueing delays. This feedback behavior differs from static optimization because the decision is expressed directly in “deadline units” rather than relying on proxy signals. The monotonicity result in Theorem 4.1 formalizes this: larger backlog can only shrink, and larger deadline budget can only expand, the Strategic-tool admission region.

8.4. Interpreting ε as an operational parameter

FTC’s conservatism is controlled by a single, interpretable parameter: the time margin ε . CADTR adds a second dimension: the priority-dependent shift $\Delta_\varepsilon^{\text{crit}}$, which controls how aggressively the system protects critical tasks. In deployment, these two parameters can be tuned independently: ε sets the baseline operating point for standard traffic, while $\Delta_\varepsilon^{\text{crit}}$ controls the safety margin for critical events.

8.5. Limitations and extensions

This paper focuses on the single-server setting to isolate the feasibility-gating principle in the simplest nontrivial queue where backlog, deadlines, and mode choice interact transparently. Multi-server, multi-GPU, or batched

extensions are important future directions. The binary critical/standard classification could be generalized to a continuous priority spectrum with graded ε -shifts. Online calibration of $\hat{s}(\cdot)$ and $\Delta_\varepsilon^{\text{crit}}$ via feedback control is another natural extension. Finally, CADTR is designed to compose with token-level and kernel-level serving optimizations by acting at an orthogonal layer: it shapes the workload by selecting among tool-permission profiles, while serving kernels optimize execution of the chosen work.

9. Conclusion

We studied deadline-constrained tool orchestration for autonomous agents through a queueing model with non-preemptive service, firm deadlines, and a multi-resolution utility structure. In this setting, tool permissioning is a systems-level control problem: enabling a tool-rich Strategic pipeline improves per-task decision quality, but it also increases congestion and the risk of catastrophic deadline failure.

We introduced *Firm-Deadline Tool Control (FTC)*, an arrival-epoch feasibility-gating policy with a provably exact monotone threshold structure in estimated backlog, deadline budget, and the time-margin parameter ε , including a generalization to arbitrarily many ordered tool tiers. We then introduced *Context-Aware Dynamic Tool Resolution (CADTR)*, which extends FTC with a minimal but consequential mechanism: event-driven dynamic risk adaptation via a priority-dependent ε -shift.

The central empirical finding is that CADTR’s semantic awareness—the ability to distinguish “this is a critical event” from “this is a routine task”—proves more valuable than the mathematical sophistication of oracle baselines that possess strictly more quantitative information. Under a nine-point load sweep with 25% critical tasks, both a Mode-Aware Oracle (exact per-task queue-wait knowledge) and a Myopic CDF Oracle (expected-utility maximization) drop up to $\sim 14\%$ of critical tasks, while CADTR maintains a practically flat $\sim 3.5\%$ critical miss rate by dynamically throttling Strategic tool usage from 41% down to 5% as load increases.

This result suggests a broader principle for the design of mission-critical AI systems: when the cost of failure varies across event types, a lightweight semantic signal (event priority) can outperform quantitatively superior but semantically blind optimization. CADTR demonstrates this principle in the simplest nontrivial setting; extending it to multi-server architectures, continuous priority spectra, and real-time deployed autonomous agents is future

work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit authorship contribution statement

Author details omitted for blinded review.

Data and Code Availability

The code to reproduce the experiments and plots is available in an anonymized repository provided for blinded review.

Appendix A. Example Prompts from the Trace Dataset

To make the FAST/SLOW distinction concrete, we list representative prompts from the trace dataset used to build the paired latency corpus. Each prompt in the dataset is executed under both a shorter-answer FAST mode and a longer-answer SLOW mode; the examples below are representative queries for which the dataset stores both outcomes.

Representative simple prompts. These are short factual, counting, or arithmetic-style queries that align naturally with the lighter FAST profile.

1. “Which ocean is on the east coast of the United States?”
2. “How many days are there in the month of April?”
3. “What is 100 divided by 4?”
4. “Which planet is closest to the Sun?”

Representative complex prompts. These are explanation-, design-, or analysis-oriented queries that are more naturally associated with the richer SLOW profile.

1. “Compare Monte Carlo methods vs deterministic numerical methods for estimating integrals or uncertainties; provide use-cases.”
2. “Design a process to ensure data quality in a large-scale data pipeline.”

3. “Design a caching strategy for a high-traffic application: what to cache, eviction policies, and how to prevent cache stampedes.”
4. “Explain how retrieval-augmented generation (RAG) can fail (hallucinations, stale context, retrieval mismatch) and propose concrete mitigations.”

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models (2023). arXiv:2210.03629, doi:10.48550/arXiv.2210.03629. URL <https://arxiv.org/abs/2210.03629>
- [2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools (2023). arXiv:2302.04761, doi:10.48550/arXiv.2302.04761. URL <https://arxiv.org/abs/2302.04761>
- [3] F. Kluge, M. Neuerburg, T. Ungerer, Utility-based scheduling of (m,k) -firm real-time task sets, in: L. M. P. Pinho, W. Karl, A. Cohen, U. Brinkschulte (Eds.), Architecture of Computing Systems – ARCS 2015, Springer International Publishing, Cham, 2015, pp. 201–211.
- [4] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, A. Tumanov, Vidur: A large-scale simulation framework for llm inference (2024). arXiv:2405.05465, doi:10.48550/arXiv.2405.05465. URL <https://arxiv.org/abs/2405.05465>
- [5] C. Nie, R. Fonseca, Z. Liu, Aladdin: Joint placement and scaling for SLO-aware LLM serving (2024). arXiv:2405.06856, doi:10.48550/arXiv.2405.06856. URL <https://arxiv.org/abs/2405.06856>
- [6] Y. Tao, Prompt-aware scheduling for low-latency llm serving (2025). arXiv:2510.03243. URL <https://arxiv.org/abs/2510.03243>

- [7] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, J. Mace, Serving DNNs like clockwork: Performance predictability from the bottom up, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020, pp. 443–462.
URL <https://www.usenix.org/conference/osdi20/presentation/gujarati>
- [8] X. Wei, Agent.xpu: Efficient scheduling of agentic llm workloads on heterogeneous socs (2025). arXiv:2506.24045.
URL <https://arxiv.org/abs/2506.24045>
- [9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), 2023, also available as arXiv:2309.06180. arXiv:2309.06180, doi:10.48550/arXiv.2309.06180.
URL <https://arxiv.org/abs/2309.06180>
- [10] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, R. Ramjee, Taming throughput-latency trade-off in llm inference with sarathi-serve (2024). arXiv:2403.02310, doi:10.48550/arXiv.2403.02310.
URL <https://arxiv.org/abs/2403.02310>
- [11] A. Patke, D. Reddy, S. Jha, C. Pinto, H. Qiu, S. Cui, C. Narayanaswami, Z. T. Kalbarczyk, R. K. Iyer, Queue management for large language model serving, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2024, also circulated as arXiv:2407.00047. arXiv:2407.00047, doi:10.48550/arXiv.2407.00047.
URL <https://arxiv.org/abs/2407.00047>
- [12] K. Hong, X. Li, L. Chen, Q. Mao, G. Dai, X. Ning, S. Yan, Y. Liang, Y. Wang, SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling, MLSys 2025 (Poster) (2025).
URL <https://mlsys.org/virtual/2025/poster/3231>

- [13] S. Chen, Z. Jia, S. Khan, A. Krishnamurthy, P. B. Gibbons, SLOs-Serve: Optimized serving of multi-SLO LLMs (2025). arXiv:2504.08784, doi:10.48550/arXiv.2504.08784.
URL <https://arxiv.org/abs/2504.08784>
- [14] W. Zhang, Jitserve: Slo-aware llm serving with imprecise request information (2025). arXiv:2504.20068.
URL <https://arxiv.org/abs/2504.20068>
- [15] Y. Tang, T. Lan, X. Huang, H. Lu, W. Chen, Scorpio: Serving the right requests at the right time for heterogeneous slo in llm inference (2025). arXiv:2505.23022.
URL <https://arxiv.org/abs/2505.23022>
- [16] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving (2024). arXiv:2401.09670, doi:10.48550/arXiv.2401.09670.
URL <https://arxiv.org/abs/2401.09670>
- [17] B. Sun, et al., Llumnix: Dynamic scheduling for large language model serving (2024). arXiv:2406.03243.
URL <https://arxiv.org/abs/2406.03243>
- [18] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, X. Jin, Fast distributed inference serving for large language models (2023). arXiv:2305.05920, doi:10.48550/ARXIV.2305.05920.
URL <https://arxiv.org/abs/2305.05920>
- [19] H. Zhang, Y. Tang, A. Khandelwal, I. Stoica, SHEPHERD: Serving DNNs in the wild, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, Boston, MA, 2023, pp. 787–808.
URL <https://www.usenix.org/conference/nsdi23/presentation/zhang-hong>
- [20] M. Luo, Autellix: An efficient serving engine for llm agents as general programs (2025). arXiv:2502.13965.
URL <https://arxiv.org/abs/2502.13965>

- [21] Y. Wang, Astraea: A state-aware scheduling engine for llm-powered agents (2025). arXiv:2512.14142.
URL <https://arxiv.org/abs/2512.14142>
- [22] Y. Wang, Augserve: Adaptive request scheduling for augmented large language model inference services (2025). arXiv:2512.04013.
URL <https://arxiv.org/abs/2512.04013>
- [23] L. Chen, M. Zaharia, J. Zou, Frugalgpt: How to use large language models while reducing cost and improving performance (2023). arXiv:2305.05176, doi:10.48550/ARXIV.2305.05176.
URL <https://arxiv.org/abs/2305.05176>
- [24] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, I. Stoica, Routellm: Learning to route llms with preference data (2024). arXiv:2406.18665, doi:10.48550/arXiv.2406.18665.
URL <https://arxiv.org/abs/2406.18665>
- [25] X. Wang, Y. Liu, W. Cheng, X. Zhao, Z. Chen, W. Yu, Y. Fu, H. Chen, MixLLM: Dynamic routing in mixed large language models, in: L. Chiruzzo, A. Ritter, L. Wang (Eds.), Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Albuquerque, New Mexico, 2025, pp. 10912–10922. doi:10.18653/v1/2025.naacl-long.545.
URL <https://aclanthology.org/2025.naacl-long.545/>
- [26] C. L. Liu, J. W. Layland, Scheduling algorithms for multiprogramming in a hard-real-time environment, *Journal of the ACM* 20 (1) (1973) 46–61. doi:10.1145/321738.321743.
- [27] J. M. López, J. L. Díaz, D. F. García, Utilization bounds for EDF scheduling on real-time multiprocessor systems, *Real-Time Systems* 28 (1) (2004) 39–68. doi:10.1023/B:TIME.0000033378.56741.14.
URL <https://doi.org/10.1023/B:TIME.0000033378.56741.14>
- [28] M. J. Neely, Stochastic Network Optimization with Application to Communication and Queueing Systems, Morgan & Claypool Publishers, 2010. doi:10.2200/S00271ED1V01Y201006CNT007.

- [29] W. Whitt, Fluid models for multiserver queues with abandonments, *Operations Research* 54 (1) (2006) 37–54. doi:10.1287/opre.1050.0227.
- [30] R. Ibrahim, W. Whitt, Real-time delay estimation based on delay history, *Manufacturing & Service Operations Management* 11 (3) (2009) 397–415. doi:10.1287/msom.1080.0223.
- [31] M. R. O. Garnett, A. Mandelbaum, Designing a call center with impatient customers, in: *Manufacturing & Service Operations Management* 4(3):208–227, 2002.
- [32] J. A. Stankovic, C. Lu, S. H. Son, G. Tao, The case for feedback control real-time scheduling, in: *Proceedings of the 11th Euromicro Conference on Real-Time Systems (ECRTS)*, 1999, pp. 11–20. doi:10.1109/EMRTS.1999.777445.
- [33] C. Lu, J. A. Stankovic, S. H. Son, G. Tao, Feedback control real-time scheduling: Framework, modeling, and algorithms, *Real-Time Systems* 23 (1–2) (2002) 85–126. doi:10.1023/A:1015398403337.
- [34] J. L. Hellerstein, Y. Diao, S. Parekh, D. M. Tilbury, *Feedback Control of Computing Systems*, Wiley-IEEE Press, 2004. doi:10.1002/047166880X.